გაკვეთილი 22 mod 3 (3.4)

გაკვეთილი 1 — დროისა და თარიღის ფუნქციები და სტრიქონული ფუნქციები

თემა: JavaScript-ში თარიღთან, დროსთან და ტექსტთან მუშაობა

გაკვეთილის მიზანი

გაკვეთილის ბოლოს მოსწავლემ უნდა შეძლოს:

- `Date` ობიექტის შექმნა;
- მიმდინარე თარიღისა და დროის მიღება;
- თარიღიდან წლის, თვის, დღის, საათის და წუთის ცალ-ცალკე მიღება;
- თარიღის ადამიანისთვის გასაგებ ფორმატში გამოტანა;
- String-ის ძირითადი თვისებებისა და მეთოდების გამოყენება;
- ტექსტის ძებნა, შეცვლა, დაყოფა და რეგისტრის შეცვლა;
- მიღებული ცოდნის ერთ პატარა პროგრამაში გაერთიანება.

---

1. დროისა და თარიღის ფუნქციები

JavaScript-ში თარიღთან და დროსთან სამუშაოდ გვაქვს ჩაშენებული `Date` **ობიექტი**.

მაგალითად, თუ გვინდა გავიგოთ მომხმარებლის მოწყობილობაზე არსებული მიმდინარე თარიღი და დრო:

```
let now = new Date();

console.log(now);
```

`new Date()` ქმნის ახალ Date ობიექტს, რომელშიც ინახება მიმდინარე თარიღი და დრო.

მაგალითად, შედეგი შეიძლება იყოს დაახლოებით:

```
Tue Oct 06 2026 19:30:15
```

აქედან ჩვენ შეგვიძლია კონკრეტული მონაცემები ცალ-ცალკე ამოვიღოთ.

`getFullYear()` — წლის მიღება

`getFullYear()` გვიბრუნებს წელს ოთხნიშნა ფორმატში.

```
let now = new Date();

console.log(now.getFullYear());
```

შედეგი:

```
2026
```

შეგვიძლია ცვლადშიც შევინახოთ:

```
let now = new Date();
let year = now.getFullYear();

console.log(year);
```

`getMonth()` — თვის მიღება

თვის მისაღებად გამოიყენება:

```
let now = new Date();

console.log(now.getMonth());
```

აქ ერთი მნიშვნელოვანი თავისებურებაა:

JavaScript-ში თვეების დათვლა იწყება 0-დან.

ანუ:

```
0  → იანვარი
1  → თებერვალი
2  → მარტი
...
11 → დეკემბერი
```

ამიტომ, თუ გვინდა თვის ჩვეულებრივი ნომერი მივიღოთ, ვუმატებთ `1`-ს:

```
let month = now.getMonth() + 1;

console.log(month);
```

`getDate()` — თვის რომელი დღეა

```
let now = new Date();

console.log(now.getDate());
```

მაგალითად, თუ დღეს 6 ოქტომბერია:

```
6
```

აქ ყურადღება მიაქციეთ განსხვავებას:

```
getDate()
```

არის **თვის რიცხვი**, ხოლო:

```
getDay()
```

არის **კვირის დღე**.

`getDay()` აბრუნებს რიცხვს `0`-დან `6`-მდე, სადაც `0` კვირაა.

`getHours()` — საათი

```
let now = new Date();

console.log(now.getHours());
```

მაგალითად:

```
19
```

`getMinutes()` — წუთი

```
let now = new Date();

console.log(now.getMinutes());
```

შეგვიძლია საათი და წუთი ერთად გამოვიტანოთ:

```
let now = new Date();

let hour = now.getHours();
let minute = now.getMinutes();

console.log(hour + ":" + minute);
```

სრული მაგალითი

```
let now = new Date();

let year = now.getFullYear();
let month = now.getMonth() + 1;
let day = now.getDate();
let hour = now.getHours();
let minute = now.getMinutes();

console.log("თარიღი: " + day + "/" + month + "/" + year);
console.log("დრო: " + hour + ":" + minute);
```

აქ უკვე Date ობიექტიდან ჩვენთვის საჭირო ინფორმაცია ამოვიღეთ და სასურველი ფორმით გამოვიტანეთ.

---

2. თარიღის ფორმატირება

ყოველთვის ხელით არ გვჭირდება წლის, თვისა და დღის აწყობა. JavaScript-ს ამისთვისაც აქვს მზა მეთოდები.

`toLocaleString()`

`toLocaleString()` თარიღსა და დროს მომხმარებლის ლოკალური ფორმატის შესაბამისად წარმოადგენს.

```
let now = new Date();

console.log(now.toLocaleString());
```

შედეგი შეიძლება იყოს დაახლოებით:

```
10/6/2026, 7:30:15 PM
```

ფორმატი დამოკიდებულია გარემოსა და ლოკალზე.

შეგვიძლია კონკრეტული ლოკალიც მივუთითოთ:

```
let now = new Date();

console.log(now.toLocaleString("ka-GE"));
```

ეს განსაკუთრებით მოსახერხებელია მაშინ, როდესაც გვინდა თარიღი მომხმარებლისთვის გასაგები ფორმით გამოვიტანოთ.

`toUTCString()`

UTC არის საერთაშორისო დროის სტანდარტი.

```
let now = new Date();

console.log(now.toUTCString());
```

მაგალითად:

```
Tue, 06 Oct 2026 15:30:15 GMT
```

მნიშვნელოვანია განსხვავება:

- `toLocaleString()` — ლოკალური დროისა და ფორმატის წარმოდგენა;
- `toUTCString()` — UTC დროის წარმოდგენა.

---

3. სტრიქონული ფუნქციები

JavaScript-ში ტექსტური მონაცემი არის **String**.

```
let text = "Hello World";
```

String-ებთან მუშაობისას ხშირად გვჭირდება გავიგოთ ტექსტის სიგრძე, შევცვალოთ ასოების ზომა, მოვძებნოთ სიტყვა, ჩავანაცვლოთ ტექსტი და ა.შ.

`length` — სტრიქონის სიგრძე

`length` გვიჩვენებს რამდენი სიმბოლოა ტექსტში.

```
let text = "Hello";

console.log(text.length);
```

შედეგი:

```
5
```

Space-იც სიმბოლოდ ითვლება:

```
let text = "Hello World";

console.log(text.length);
```

შედეგია `11`.

> `length` თვისებაა და არა მეთოდი, ამიტომ `()` არ სჭირდება.

---

4. `toUpperCase()` და `toLowerCase()`

`toUpperCase()` ყველა ასოს დიდ ასოდ გარდაქმნის:

```
let text = "hello world";

```

```
console.log(text.toUpperCase());
```

შედეგი:

```
HELLO WORLD
```

`toLowerCase()` კი პირიქით:

```
let text = "HELLO WORLD";

console.log(text.toLowerCase());
```

შედეგი:

```
hello world
```

ეს ძალიან გამოსადეგია, როდესაც მომხმარებლის შეყვანილი ტექსტების შედარება გვინდა.

მაგალითად:

```
let answer = "YES";

if (answer.toLowerCase() === "yes") {
    console.log("Correct");
}
```

ამ შემთხვევაში `YES`, `Yes` და `yes` შეგვიძლია ერთნაირად დავამუშაოთ.

---

5. `includes()` — ტექსტის მოძებნა

`includes()` ამოწმებს, შეიცავს თუ არა სტრიქონი კონკრეტულ ტექსტს.

```
let text = "I am learning JavaScript";

console.log(text.includes("JavaScript"));
```

შედეგი:

```
true
```

ხოლო:

```
console.log(text.includes("Python"));
```

მიიღებს:

```
false
```

შეგვიძლია `if`-თან ერთად გამოვიყენოთ:

```
let email = "giorgi@gmail.com";

if (email.includes("@")) {
    console.log("Email contains @");
} else {
    console.log("Invalid email");
}
```

---

6. `indexOf()`

`indexOf()` გვეუბნება, რომელი პოზიციიდან იწყება კონკრეტული ტექსტი.

```
let text = "Hello World";

console.log(text.indexOf("World"));
```

შედეგი:

```
6
```

პოზიციების დათვლა იწყება `0`-დან:

```
H e l l o   W o r l d
0 1 2 3 4 5 6 7 8 9 10
```

თუ სიტყვა ვერ იპოვა:

```
console.log(text.indexOf("Java"));
```

შედეგი იქნება:

```
-1
```

---

7. `replace()` — ტექსტის ჩანაცვლება

```
let text = "I like JavaScript";

let newText = text.replace("JavaScript", "TypeScript");

console.log(newText);
```

შედეგი:

```
I like TypeScript
```

თავდაპირველი `text` ავტომატურად არ იცვლება. შეცვლილი შედეგი აქ `newText` ცვლადში შევინახეთ.

---

8. `slice()` — ტექსტის ნაწილის ამოღება

`slice()` საშუალებას გვაძლევს სტრიქონიდან მხოლოდ კონკრეტული ნაწილი ავიღოთ.

```
let text = "JavaScript";

console.log(text.slice(0, 4));
```

შედეგი:

```
Java
```

ანუ:

```
slice(0, 4)
```

იწყებს `0` ინდექსიდან და იღებს სიმბოლოებს `4` პოზიციამდე — ბოლო პოზიცია არ შედის.

9. `split()` — სტრიქონის დაყოფა

`split()` ტექსტს ნაწილებად ყოფს.

```
let text = "HTML,CSS,JavaScript";

let languages = text.split(",");

console.log(languages);
```

შედეგად მივიღებთ მასივს:

```
["HTML", "CSS", "JavaScript"]
```

მაგალითად, წინადადება შეგვიძლია სიტყვებად დავყოთ:

```
let text = "I love JavaScript";

let words = text.split(" ");

console.log(words);
```

---

10. `trim()` — ზედმეტი Space-ების მოშორება

ხშირად მომხმარებელი ფორმაში ტექსტს შემთხვევით Space-ებით წერს.

```
let username = "   Giorgi   ";

console.log(username.trim());
```

შედეგი:

```
Giorgi
```

`trim()` აშორებს ზედმეტ ცარიელ ადგილებს **სტრიქონის დასაწყისიდან და ბოლოდან**.

---

11. Date + String ერთად

ახლა ორივე თემა ერთ პატარა მაგალითში გავაერთიანოთ.

გვინდა პროგრამამ გამოიტანოს მომხმარებლის სახელი დიდი ასოებით და მიმდინარე თარიღი:

```
let username = "giorgi";
let now = new Date();

let message =
    "Hello, " +
    username.toUpperCase() +
    "! Today is " +
    now.toLocaleString();

console.log(message);
```

შეგვიძლია Template Literal-იც გამოვიყენოთ:

```
let username = "giorgi";
let now = new Date();

console.log(`Hello, ${username.toUpperCase()}! Today is
${now.toLocaleString()}`);
```

ეს კარგი მაგალითია იმის საჩვენებლად, რომ რეალურ პროგრამაში სხვადასხვა ფუნქცია ერთმანეთისგან იზოლირებულად არ გამოიყენება.

პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — მიმდინარე თარიღი

შექმენით პროგრამა, რომელიც გამოიტანს:

```
Year: 2026
Month: 10
Day: 6
```

გამოიყენეთ:

```
new Date()
getFullYear()
getMonth()
getDate()
```

სავარჯიშო 2 — საათი

დაწერეთ პროგრამა, რომელიც მიმდინარე საათსა და წუთს გამოიტანს ფორმატით:

```
Current time: 19:35
```

გამოიყენეთ:

```
getHours()
getMinutes()
```

სავარჯიშო 3 — ტექსტის დამუშავება

მოცემულია:

```
let text = "JavaScript is interesting";
```

პროგრამამ უნდა:

1. გამოიტანოს ტექსტის სიგრძე;
2. გადააქციოს მთლიანად დიდ ასოებად;
3. შეამოწმოს შეიცავს თუ არა `"JavaScript"`-ს;
4. `"JavaScript"` ჩაანაცვლოს `"TypeScript"`-ით.

სავარჯიშო 4 — მომხმარებლის სახელი

მოცემულია:

```
let username = "   nino   ";
```

მიიღეთ შედეგი:

```
NINO
```

ანუ მოსწავლემ თავად უნდა გადაწყვიტოს, როგორ გამოიყენოს:

```
trim()
toUpperCase()
```

ერთად.

სავარჯიშო 5 — სიტყვების მასივი

მოცემულია:

```
let technologies = "HTML CSS JavaScript React";
```

`split()`-ის გამოყენებით გადააქციეთ მასივად:

```
["HTML", "CSS", "JavaScript", "React"]
```

შედარებით რთული პრაქტიკული დავალება

მოცემულია:

```
let name = "   giorgi   ";
let course = "I am learning JavaScript";
```

შექმენით პროგრამა, რომელმაც უნდა:

- `name`-ს მოაშოროს ზედმეტი Space-ები;
- სახელი გადაიყვანოს დიდ ასოებში;
- შეამოწმოს, შეიცავს თუ არა `course` სიტყვას `"JavaScript"`;
- მიიღოს მიმდინარე წელი;
- მიიღოს მიმდინარე საათი;
- საბოლოოდ გამოიტანოს დაახლოებით:

```
Hello GIORGI!
You are learning JavaScript.
Current year: 2026
Current hour: 19
```